

▮ CS50x Lecture 1: C Basics

Brief Overview

This note covering C programming fundamentals was created from the [CS50x 2026 - Lecture 1 - C](#) YouTube video. It dives into functions, variables, loops, conditionals, and compiling, giving you the essential building blocks for writing, building, and running C programs.

Key Points

- Learn how to define and call **functions** that perform reusable tasks.
- Understand the role of a **compiler** and the typical build steps with make.
- Explore core concepts like variables, loops, and conditional statements.
- Get hands-on examples from a live lecture to reinforce syntax and workflow.

▮ From Scratch to C: Core Programming Concepts

- **Functions** – encapsulate a reusable piece of work.
- **Variables** – storage locations identified by names.
- **Loops** – repeat actions until a condition is met.
- **Conditionals** – decide which code path to follow based on a Boolean test.

In Scratch these ideas appear as colored puzzle blocks; in C they appear as typed syntax.

▮ Source Code vs. Machine Code

Aspect	Source Code	Machine Code
Who reads it?	Humans (with training).	Computer processor (binary).
Typical format	Text files (.c, .py, etc.).	Sequences of 0 and 1 .
Example	<code>printf("Hello, world!\n");</code>	01001101 01100101 ...
Conversion tool	Compiler (e.g., make).	–

Source code is the programmer-written input; the compiler translates it into machine code that the hardware executes.

▮ Compilers and the Build Process

- **Compiler** – software that takes source code (C) and produces machine code (executable).
- **make** – a common front-end that invokes the compiler, inferring the source file (hello.c) from the target name (hello).
- **Build steps:**
 1. Write source (hello.c).
 2. Run make hello.

3. Execute with `./hello`.

If the compiler reports no output, the compilation succeeded.

▮ Visual Studio Code (VS Code) Overview

- **GUI (Graphical User Interface)** – windows, icons, menus.
- **CLI (Command-Line Interface)** – the terminal at the bottom, indicated by a \$ prompt.
- **Key features:**
 - **Syntax highlighting** – colors different language tokens automatically.
 - **File Explorer** – left pane showing project files.
 - **Activity Bar** – modern menu for accessing tools (often hidden for focus).

VS Code combines a GUI for editing with a CLI for building and running programs.

▮ Writing & Compiling a Simple C Program

```
#include

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

- **#include** – imports the standard I/O library.
- **int main(void)** – entry point function returning an integer.
- **printf** – prints formatted text to the console.
- **"\n"** – escape sequence for a new line.
- **;** – statement terminator.

Commands

1. `code hello.c` – creates/opens the file in VS Code.
2. `make hello` – compiles `hello.c` into the executable `hello`.
3. `./hello` – runs the program, displaying `Hello, world!`.

⚠ Common Errors & Debugging

Error Type	Typical Symptom	Fix
Missing semicolon	Compiler error: <i>expected ';' after expression</i>	Add ; at the end of the statement.
Omitted <code>#include</code>	“implicit declaration of function ‘printf’”	Insert the include line at the top.

Forgetting <code>\n</code>	Prompt appears on same line as output	Add <code>\n</code> inside the string or use an appropriate escape sequence.
Not recompiling after edit	Output unchanged	Re-run make hello before executing.

The compiler points to the file name, line number, and character position to help locate the mistake.

▮ Escape Sequences in C

Sequence	Meaning
<code>\n</code>	New line (line feed).
<code>\r</code>	Carriage return (returns cursor to line start).
<code>\"</code>	Literal double-quote inside a string.
<code>\\</code>	Literal backslash.

- Used when the literal character would otherwise terminate the string or have special meaning.
- Example: `printf("She said, \"Hello!\\n\")`; prints:

```
She said, "Hello!"
```

▮ Terminology Quick Reference

GUI – *Graphical User Interface*: visual elements like windows and icons.

CLI – *Command-Line Interface*: text-based input where commands are typed.

Compiler – *Program that translates source code to machine code.*

Syntax highlighting – *Color-coding of language constructs to improve readability.*

▮ Header Files & Libraries

Header file – a file ending in `.h` that contains declarations (function prototypes, macros, type definitions) provided by a library.

Library – compiled code (usually `.a` or `.so`) that implements functions you can reuse without writing them yourself.

- The C standard library is accessed via header files such as `#include` .
- Third-party or course-specific libraries (e.g., CS50's helpers) are accessed with `#include "cs50.h"`.
- Missing a required header causes errors like **“undeclared library function printf”**.

How to Include a Header

```
#include // standard I/O definitions (printf, scanf, ...)
#include "cs50.h" // CS50 convenience functions (get_string, ...)
```

- Angle brackets <> tell the compiler to search system include directories.
- Double quotes " " direct the search to the current project folder first, then fall back to system paths.

▮ Using CS50's Helper Functions

Function	Purpose	Typical Return Type
get_string(prompt)	Prompt the user and read an entire line of text	string (alias for char *)
get_int(prompt)	Prompt and read an integer	int
get_double(prompt)	Prompt and read a floating-point number	double

Note – These helpers are built on top of the standard library; they internally call printf to display the prompt.

Example: Getting a Name

```
string name = get_string("What's your name? ");
```

- The literal text inside the quotes is printed exactly as shown.
- The trailing space keeps the cursor on the same line as the user's input.

▮ Formatting Output with printf

printf – prints formatted text to the standard output stream.

- **Placeholders** (format specifiers) indicate the type of each argument that follows the format string.
- %s inserts a *string*; %d inserts an *int*; %f inserts a *floating-point* value.

```
printf("Hello, %s\n", name);
```

- The comma after the format string separates the first argument (the format) from subsequent arguments (the values to substitute).
- \n is the *newline* escape sequence that moves the cursor to the next line.

Common Pitfalls

- Forgetting the placeholder (%s) results in the literal characters being printed (e.g., printf("Hello, answer")).
- Omitting the comma between the format string and the variable causes a syntax error.

▮ Command-Line Interface (CLI) Basics

Command	Description	Example
ls	List files in the current directory	ls
mkdir	Create a new directory	mkdir hello
rm	Delete a file	rm hello
rm -r	Recursively delete a directory	rm -r old_folder
mv	Move or rename a file/directory	mv hello.c hello/
cd	Change the current directory	cd hello
cp	Copy a file	cp hello.c backup.c

- Commands are **short** to speed up navigation; they operate on the *current working directory* shown in the terminal prompt.
- Most commands accept *options* (e.g., -r for recursive removal).

▮ Operating Systems Overview

Operating system (OS) – software that manages hardware resources (CPU, memory, I/O) and provides services to applications.

- **GUI** (graphical) vs. **CLI** (text-based) interfaces: the former offers windows and icons, the latter offers rapid keyboard-driven control.
- **Linux** – a widely used, open-source OS derived from **Unix**; popular on servers and in development environments.
- VS Code runs on macOS, Windows, and Linux; the underlying server (e.g., cs50.dev) hosts pre-installed header files (stdio.h, cs50.h) so the compiler can locate them automatically.

▮ Building & Organizing Projects

1. **Write source files** (*.c).
2. **Compile** with make → produces an executable ().
3. **Run** the executable: ./.

Keeping the Workspace Clean

- Remove old executables: rm (e.g., rm hello).
- Organize sources into folders:

```
mkdir hello      # create a folder named "hello"
mv hello.c hello/ # move the source file into that folder
```

- Use ls to verify file locations before compiling.

▮ Reference Resources

- [Manual pages](#) (man) – the authoritative documentation for C functions and system commands.
 - [CS50's curated manual](#) – simplified versions of the official pages at manual.cs50.org.
 - When an error mentions a missing header, consult the manual to discover the required `#include` directive.
-

▮ Key Takeaways

- [Header files](#) give your program access to external code (libraries).
- [printf](#) uses format specifiers (e.g., `%s`) to embed variable values into output strings.
- [get_string](#) (and other CS50 helpers) streamline user input, automatically handling prompts.
- Mastering [CLI commands](#) (`ls`, `mkdir`, `mv`, `rm`, `cd`) is essential for efficient file management.
- Understanding the role of the [operating system](#) clarifies why certain files are pre-installed and how the compiler finds them.

▮ Navigating Files & Directories

- [ls](#) – list the contents of the current directory.
- [cd](#)

– change the working directory to

.

Directory: a folder that groups files; the path tells the OS where the folder resides.

- [cd ..](#) – move up one level (the *parent* directory).
- [cd .](#) – stay in the current directory (the *dot* refers to “this folder”).

When you type `ls` after creating a folder called [hello](#), you’ll see:

```
hello/
```

The trailing slash indicates that [hello](#) is a directory.

▮ Managing Files

Command	Purpose	Example
<code>mv</code>	Move or rename a file	<code>mv hello.c old.c</code> (rename)
<code>cp</code>	Copy a file	<code>cp hello.c backup.c</code>
<code>rm</code>	Delete a file (prompts for confirmation)	<code>rm hello</code>
<code>rm -r</code>	Delete a directory recursively	<code>rm -r hello</code>

Rename vs. Move – mv serves both roles; providing two filenames causes a rename, while giving a path causes a move.

▮ Working with Folder Hierarchies

- **Parent (..)** – refers to the directory one level above.
- **Current (.)** – refers to the directory you are in.

To move **hello.c** out of its folder into the parent directory:

```
mv hello.c ..
```

After the move, the file appears at the top-left of the VS Code explorer, no longer indented under **hello/**.

▮ Building & Rebuilding Programs in Subfolders

1. Create a folder (e.g., mkdir hello).
2. Move the source file into it (mv hello.c hello/).
3. Change into the folder (cd hello).
4. Compile locally: make hello.
5. Run the executable: ./hello.

If you later delete the executable with rm hello and re-compile, both the source file and the new executable reappear inside the same folder.

To return to your home (default) directory from anywhere, simply press **Enter** after typing cd with no arguments:

```
cd
```

▮ Conditional Logic in C

Basic if statement

```
if (x < y) {  
    printf("x is less than y\n");  
}
```

- The parentheses contain a **boolean expression** (evaluates to true or false).
- Curly braces {} group the statements that run when the condition is true.

if... else chain

```
if (x < y) {  
    printf("x is less than y\n");  
} else if (x > y) {  
    printf("x is greater than y\n");  
} else {  
    printf("x is equal to y\n");  
}
```

Equality operator (==) – compares two values; a single = performs assignment.

Design tip

When only three mutually exclusive possibilities exist (less, greater, equal), checking the first two conditions is sufficient; the final else implicitly covers the remaining case.

▮ Incrementing and Decrementing Variables

Form	Meaning
counter = counter + 1;	Assigns the current value plus one back to counter.
counter += 1;	Shorthand for the above.
counter++;	Increments counter by one (post-increment).

Decrementing uses the minus sign (counter-- or counter -= 1).

▮ Data Types Overview

Type	Typical size	Example values
bool	1 byte	true, false
char	1 byte	'A', 'z'
int	32 bits	-2 147 483 648 ... 2 147 483 647
long	64 bits (on most modern systems)	Larger integer range
float	32 bits	3.14f
double	64 bits	3.1415926535

Floating-point types can represent only a finite set of real numbers; double provides more precision than float but still cannot capture every possible real value.

▮ Formatting Output Placeholders

When using printf, the format specifier tells the compiler what type to expect:

Specifier	Data type
%c	single character (char)
%s	string (char *)
%d or %i	integer (int)
%f	floating-point (float or double)
%li	long integer (long)

Example:

```
printf("Value: %f\n", x);
```

▮ Putting It All Together – A Simple Compare Program

```
#include
#include

int main(void)
{
    int x = get_int("What's X? ");
    int y = get_int("What's Y? ");

    if (x < y) {
        printf("X is less than Y\n");
    } else if (x > y) {
        printf("X is greater than Y\n");
    } else {
        printf("X is equal to Y\n");
    }
}
```

- `get_int` (from `cs50.h`) reads an integer from the user.
- The program demonstrates directory navigation, compilation (`make compare`), and the conditional flow discussed earlier.

▮ Refining Conditional Logic

Conditional statement – a piece of code that chooses a path based on a Boolean expression.

- When several mutually exclusive tests are needed, use an `if ... else if ... else` chain.
- Avoid **redundant checks**: after confirming `x < y` or `x > y`, the remaining case is automatically `x == y`.
- Pruning unnecessary tests makes the flow chart tighter and the compiled program shorter.

Logical Operators

Operator	Meaning	Typical use
,		,
&&	Logical AND – true only if <i>both</i> operands are true.	if (a > 0 && b > 0)
!	Logical NOT – inverts a Boolean value.	if (!valid)

Important: Using && between a lower-case and upper-case character (c == 'y' && c == 'Y') can never be true, so it would be a bug.

```
char c = get_char("Do you agree? ");
if (c == 'y' || c == 'Y')
{
    printf("Agreed.\n");
}
else if (c == 'n' || c == 'N')
{
    printf("Not agreed.\n");
}
```

□ Character vs. String Input

Character literal – a single character enclosed in *single* quotes (e.g., 'A').

String literal – a sequence of characters enclosed in *double* quotes (e.g., "Hello").

- get_char returns a **char**, ideal for one-letter responses (Y/N).
- get_string returns a **string** (char *), used for full words or sentences.

```
char answer = get_char("Enter Y or N: ");
if (answer == 'Y')
{
    printf("You said Yes.\n");
}
```

□ Loop Constructs in C

Loop – repeats a block of code while a Boolean condition remains true.

While Loops

- **Count down** (starts at a positive number, decrements until 0):

```
int i = 3;
while (i > 0)
{
    printf("meow\n");
    i--;          // same as i = i - 1;
}
```

- **Count up** (starts at 0, increments while < limit):

```
int i = 0;
while (i < 3)
{
    printf("meow\n");
    i++;          // same as i = i + 1;
}
```

For Loops

- Compact form that combines initialization, condition, and iteration in one line.

```
for (int i = 0; i < 3; i++)
{
    printf("meow\n");
}
```

Style	Typical start	End condition	Increment
while (count up)	i = 0	i < n	i++
while (count down)	i = n	i > 0	i--
for	int i = 0	i < n	i++

- **Best practice:** start at 0 and loop while i < n; this makes the range clear and avoids off-by-one errors.

Infinite Loop & Control

- An intentionally endless loop uses a constant true condition:

```
while (true)
{
    printf("meow\n");
}
```

- **Terminate:** press **Ctrl+C** in the terminal.
 - Inside any loop, two special statements can alter flow:
 - continue; – skip the rest of the current iteration and re-evaluate the condition.
 - break; – exit the loop immediately.
-

□ Input Validation & Variable Scope

Using `get_int` for a Repetition Count

- `get_int` repeatedly prompts until the user supplies a valid integer.
- After obtaining the number, validate that it is non-negative before using it in a loop.

```
int n;
while (true)
{
    n = get_int("How many times? ");
    if (n < 0)
    {
        // Negative input is rejected; ask again.
        continue;
    }
    break;          // Valid input; exit the validation loop.
}

/* Use the validated value to control a for-loop */
for (int i = 0; i < n; i++)
{
    printf("meow\n");
}
```

Variable Scope

Scope – the region of a program where a declared identifier (e.g., a variable) is visible.

- Declaring a variable **inside** a block (between `{` and `}`) limits its visibility to that block.
- Declaring **outside** the block makes it available to all subsequent code.

Declaration location	Visibility
Inside loop <code>{ ... }</code>	Only within that loop (cannot be used after the loop).
Before loop (outside)	Throughout the function, including after the loop.

- Example of a scope error (undeclared identifier) and its fix:

```
/* Incorrect – N declared inside the while loop */
while (true)
{
    int N = get_int("Enter N: ");    // N exists only here
    if (N >= 0) break;
}
printf("%d\n", N);    // □ compilation error: N not declared

/* Correct – declare N before the loop */
int N;
while (true)
{
    N = get_int("Enter N: ");
```

```
    if (N >= 0) break;
}
printf("%d\n", N);    // □ works
```

□ Building a Dynamic “Meow” Program

1. **Prompt** the user for a repeat count with `get_int`.
2. **Validate** the count (reject negatives).
3. **Loop** using a `for` (or `while`) construct to print “meow” the requested number of times.

The resulting program combines the concepts covered earlier:

- **Input functions** (`get_int`, `get_char`).
- **Conditional checks** (`if`, logical operators).
- **Loops** (`while`, `for`).
- **Proper variable scope** (declare `n` before the validation loop).

Following these patterns yields clean, maintainable C code that mirrors the visual flow of Scratch blocks while exploiting the power of a compiled language.

□ Loop Control: `break` & `continue` ⚙️

- **`break`**; – exits the innermost loop immediately.
- **`continue`**; – skips the rest of the current iteration and reevaluates the loop condition.

Both statements are optional; a loop can be written using only one of them (or none) if the logic permits.

Typical pattern for validating input

```
while (true)
{
    int N = get_int("How many times? ");
    if (N >= 0)
    {
        break;           // stop looping once a non-negative number is entered
    }
}
```

- The loop runs **infinitely** until the `break` condition is satisfied.
- Using `continue` would be useful when you want to **ignore** a particular iteration (e.g., when `N < 0`) and let the loop try again.

□ `do ... while` Loop □

- Syntax places the **condition after** the loop body, guaranteeing that the body executes **at least once**.

```
int N;
do
{
    N = get_int("How many times? ");
}
while (N < 0);
```

- **Difference from while:** while (condition) checks the condition **before** the first iteration; do ... while checks **after**.
- Ideal for **input-prompt** scenarios where you must ask the user at least once.

▮ Function Design: Prototypes, Definitions, & Scope

1. Function Prototype (aka *signature*)

Placed **above** main (or any calling code) to inform the compiler of a function that will be defined later.

```
void meow(void);           // no arguments, no return value
```

- Ends with a **semicolon**.
- Serves as a *promise* that a matching definition exists somewhere in the file.

2. Function Definition

```
void meow(void)
{
    printf("meow\n");
}
```

- No semicolon after the header line (only after the prototype).
- The body is enclosed in curly braces { ... }.

3. Functions with Parameters

```
void meow(int times)
{
    for (int i = 0; i < times; i++)
    {
        printf("meow\n");
    }
}
```

- Parameter type and name are declared inside the parentheses.
- The parameter name (times) is **local** to the function; it does **not** affect variables of the same name elsewhere.

4. Functions that Return a Value

```
int get_nonnegative_int(void)
{
    int n;
    do
    {
        n = get_int("Enter a non-negative integer: ");
    } while (n < 0);
    return n;
}
```

- return passes the computed value back to the caller.
- The caller can store the result: `int N = get_nonnegative_int();`

▮ Abstracting Repetition with Functions

- **Goal:** Replace repeated `printf("meow\n");` statements with a single, reusable function.
- **First version:** `void meow(void)` – hard-coded to print once.
- **Improved version:** `void meow(int times)` – prints *times* repetitions, eliminating duplicated loops in main.

Abstraction lets you change the implementation (e.g., switch from printing to playing a sound) without touching the rest of the program.

▮ Variable Scope & Lifetime

Variable Declaration	Scope (where it's visible)
Inside a function (e.g., <code>int N;</code> in main)	From the point of declaration to the closing <code>}</code> of that block
Inside a loop block (for / while)	Only within that loop's <code>{ }</code>
As a function parameter (<code>int times</code>)	Only inside the function body

- Declaring a variable **outside** of any block (e.g., at file level) gives it *global* scope, but the notes focus on *local* scope.
- Attempting to use a variable **outside** its scope produces an “**undeclared identifier**” error (as seen when `N` was referenced after the block where it was defined).

▮ Comments & Documentation

- **Single-line comment:** starts with `//` and is ignored by the compiler.

```
// This line explains the purpose of the following code
```

- Used for **human-readable notes**, similar to Scratch's sticky notes.

- Removing comments does **not** affect program behavior.

▮ Simple Text-Based Game Example (Mario) ▮

Objective

Print a row of question-mark symbols (?) using a loop, laying groundwork for a text-based version of *Super Mario Bros.*

Minimal Implementation (hard-coded)

```
#include

int main(void)
{
    printf("????\n");
    return 0;
}
```

Loop-Based Implementation

```
#include

int main(void)
{
    int count = 4;                // number of question marks
    for (int i = 0; i < count; i++)
    {
        printf("?");              // print one question mark each iteration
    }
    printf("\n");                 // final newline
    return 0;
}
```

- Changing count modifies how many symbols appear without altering the loop logic.
- Demonstrates **iteration**, **variable reuse**, and **output formatting**—core building blocks for more complex game logic (e.g., moving Mario, generating obstacles).

▮ Design, Correctness, & Style Checklist ▮

Aspect	What to verify
Correctness	Program compiles (make) and runs as intended.
Design	Functions are modular (single responsibility) and reusable .
Style	• Consistent indentation • Meaningful variable names (times, count)

- Required comments (//) for non-obvious logic

- **Tools:** check50 (correctness), design50 (design feedback), style50 (style enforcement).
- Applying these tools iteratively leads to code that is *functional, well-designed, and readable*.

□ Key Takeaways

Scope determines visibility; prototypes inform the compiler; do-while guarantees at least one execution; breaking loops early simplifies logic; abstracting repeated work into functions improves design; and even simple console output can be the foundation of a larger game.

□ Newline Placement & Row vs. Column

`\n` – escape sequence that moves the cursor to a new line.

- When printing a series of characters in a **row**, place `\n` **after** the loop finishes.
- When printing a **column**, put `\n` **inside** the loop after each character.

```
/* Row of four question marks */
for (int i = 0; i < 4; i++)
{
    printf("?");
}
printf("\n");    // newline after the whole row
```

```
/* Column of three hashes */
for (int i = 0; i < 3; i++)
{
    printf("#\n");    // newline after each hash
}
```

□ Nested Loops for 2-D Output

Nested loops – an outer loop controlling rows and an inner loop controlling columns.

- **Outer loop** iterates over rows.
- **Inner loop** iterates over columns for each row.

```
for (int row = 0; row < 3; row++)        // three rows
{
    for (int col = 0; col < 3; col++)    // three columns per row
    {
        printf("#");                    // print one brick
    }
}
```

```
printf("\n");           // end of current row
}
```

Result: a 3×3 grid of # characters.

▮ Variable Naming & Scope in Nested Loops

- Use **different identifiers** for the outer and inner loop counters (e.g., row and col).
- Each counter's **scope** is limited to its own for statement block, preventing accidental reuse.

Scope level	Visible identifier
Inside inner loop	col (and row from outer)
After inner loop, before next iteration of outer	row only
Outside both loops	Neither row nor col

▮ Constants vs. Variables

const – keyword that makes a variable immutable after its initial definition.

```
const int N = 3;    // N cannot be reassigned later
```

- Using a constant avoids “magic numbers” scattered throughout the code.
- Changing the grid size later only requires editing the single declaration of N.

▮ Arithmetic Operators Cheat Sheet

Operator	Meaning	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division (integer or floating-point)	x / y
%	Modulo (remainder)	$x \% y$

▮ Building a Simple Calculator

Addition

```
#include
#include

int main(void)
{
    int x = get_int("X? ");
    int y = get_int("Y? ");
    int sum = x + y;
    printf("%d\n", sum);
}
```

- Optional: omit sum and print $x + y$ directly.

Multiplication (doubling)

```
int x = get_int("X? ");
printf("%d\n", x * 2);
```

□ Loop-Based Doubling Game & Integer Overflow

```
#include
#include

int main(void)
{
    int dollars = 1;
    while (true)
    {
        char c = get_char("Here $%d. Double it and give to next person? (y/n) ", dollars);
        if (c == 'y' || c == 'Y')
        {
            dollars *= 2;        // double the amount
        }
        else
        {
            break;              // exit the infinite loop
        }
    }
}
```

- The program demonstrates **exponential growth** ($\text{dollars} = \text{dollars} * 2$).
- When dollars exceeds the maximum value of a 32-bit int ($\sim 2 \times 10^9$), **integer overflow** occurs, causing the value to wrap around to a negative number.

□ Integer vs. Long & Overflow Limits

- int $\rightarrow$ usually 32 bits $\rightarrow$ max $\approx 2\,147\,483\,647$.

- long → typically 64 bits → $\max \approx 9 \times 10^{18}$.

```
long dollars = 1L;           // use long literal
printf("%ld\n", dollars);
```

- Even a 64-bit long will eventually overflow if the doubling loop runs long enough.

▮ Truncation & Floating-Point Precision

- Dividing two ints performs **integer division**, discarding the fractional part.

```
int x = 1, y = 3;
printf("%d\n", x / y);  // prints 0
```

- Use float or double for fractional results, and the appropriate format specifier.

```
float x = 1.0f, y = 3.0f;
printf("%f\n", x / y);  // prints 0.333333
```

- Precision can be limited with a *field width*: `printf("%.2f\n", value);` shows two digits after the decimal point.
- Because floating-point numbers are stored with a finite number of bits, some values (e.g., $1/3$) can only be **approximated**, leading to small rounding errors.

▮ Historical Integer-Overflow Bugs

Event	Cause	Consequence
Y2K (year 2000)	Years stored with two digits	Potential misinterpretation of 2000 as 1900
2038 problem	32-bit time counters counting seconds since 1 Jan 1970	After 19 January 2038 counters overflow, yielding dates around 1901
Boeing 787 GCU bug	32-bit counter overflow after 248 days of continuous power	Loss of AC power; temporary fix: reboot the aircraft
Pac-Man level-256 glitch	Integer overflow in level counter	Screen garble at level 256

- **Mitigation strategies**: use larger integer types, regularly reset counters, or redesign algorithms to avoid unbounded counting.